

tf.compat.v1.transpose Stay organized with collections Save and categorize content based on your preferences.

https://www.tensorflow.org/api_docs/python/tf/compat/v1/transpose

Transposes a.

Function Signatures

```
tf.compat.v1.transpose( a, perm=None, name='transpose',  
conjugate=False )
```

Code Examples

```
tf.compat.v1.transpose(  
a, perm=None, name='transpose', conjugate=False  
)
```

```
tf.compat.v1.transpose(  
a, perm=None, name='transpose', conjugate=False  
)
```

```

x = tf.constant([[1, 2, 3], [4, 5, 6]])
tf.transpose(x) # [[1, 4]
# [2, 5]
# [3, 6]]
# Equivalently
tf.transpose(x, perm=[1, 0]) # [[1, 4]
# [2, 5]
# [3, 6]]
# If x is complex, setting conjugate=True gives the conjugate
transpose
x = tf.constant([[1 + 1j, 2 + 2j, 3 + 3j],
[4 + 4j, 5 + 5j, 6 + 6j]])
tf.transpose(x, conjugate=True) # [[1 - 1j, 4 - 4j],
# [2 - 2j, 5 - 5j],
# [3 - 3j, 6 - 6j]]
# 'perm' is more useful for n-dimensional tensors, for n > 2
x = tf.constant([[[ 1, 2, 3],
[ 4, 5, 6]],
[[ 7, 8, 9],
[10, 11, 12]]])
# Take the transpose of the matrices in dimension-0
# (this common operation has a shorthand `linalg.matrix_transpose`)
tf.transpose(x, perm=[0, 2, 1]) # [[[1, 4],
# [2, 5],
# [3, 6]],
# [[7, 10],
# [8, 11],
# [9, 12]]]

```

```

x = tf.constant([[1, 2, 3], [4, 5, 6]])
tf.transpose(x) # [[1, 4]
# [2, 5]
# [3, 6]]
# Equivalently
tf.transpose(x, perm=[1, 0]) # [[1, 4]
# [2, 5]
# [3, 6]]
# If x is complex, setting conjugate=True gives the conjugate
transpose
x = tf.constant([[1 + 1j, 2 + 2j, 3 + 3j],
[4 + 4j, 5 + 5j, 6 + 6j]])
tf.transpose(x, conjugate=True) # [[1 - 1j, 4 - 4j],
# [2 - 2j, 5 - 5j],
# [3 - 3j, 6 - 6j]]
# 'perm' is more useful for n-dimensional tensors, for n > 2
x = tf.constant([[[ 1, 2, 3],
[ 4, 5, 6]],
[[ 7, 8, 9],
[10, 11, 12]]])
# Take the transpose of the matrices in dimension-0
# (this common operation has a shorthand `linalg.matrix_transpose`)
tf.transpose(x, perm=[0, 2, 1]) # [[[1, 4],
# [2, 5],
# [3, 6]],
# [[7, 10],
# [8, 11],
# [9, 12]]]

```

Documentation

Transposes a.

Permutes the dimensions according to perm.

The returned tensor's dimension i will correspond to the input dimension $\text{perm}[i]$. If perm is not given, it is set to $(n-1 \dots 0)$, where n is the rank of the input tensor. Hence, by default, this operation performs a regular matrix transpose on 2-D input Tensors. If conjugate is True and `a.dtype` is either `complex64` or `complex128` then the values of `a` are conjugated and transposed.

For example:

`## Returns`

numpy compatibility

In numpy transposes are memory-efficient constant time operations as they simply return a new view of the same data with adjusted strides.

TensorFlow does not support strides, so transpose returns a new tensor with the items permuted.